

Лабораторная работа № 5

Тема: Работа с файловой системой — асинхронная.

Общие требования:

- Использовать модуль `fs.promises` (или `require('fs/promises')`).
- Весь код должен быть обернут в асинхронные функции (`async function`).
- Обязательное использование `try...catch` для обработки ошибок.
- Использовать `path.join(__dirname, ...)` для путей.

Вариант №1: "Асинхронный архиватор логов"

Задание 1: Создание структуры. Напишите асинхронную функцию `setup()`, которая проверяет наличие папки `storage`. Если её нет — создает её. После создания запишите в файл `storage/status.txt` строку "Система готова".

Задание 2: Асинхронное чтение. Создайте файл `tasks.txt` вручную. Напишите функцию, которая асинхронно читает этот файл. Если файл прочитан успешно — выведите его содержимое в верхнем регистре. Если файла нет — выведите "Файл задач не найден", не роняя программу.

Задание 3: Логирование с меткой времени. Реализуйте функцию `addLog(message)`, которая асинхронно дописывает (`append`) в файл `storage/activity.log` строку с текущей датой и переданным сообщением.

Задание 4: Цепочка операций. Напишите функцию, которая:

- Читает файл `source.txt`.
- Создает копию этого файла с названием `copy.txt`.
- После успешного копирования удаляет оригинал (`source.txt`).

Задание 5: Параллельная проверка (`Promise.all`). Дополнительное задание: Создайте массив имен файлов. Используя `Promise.all` и метод `fs.access` (или `fs.stat`), проверьте существование всех этих файлов одновременно и выведите результат.

Вариант №2: "Асинхронный менеджер профилей"

Задание 1: Инициализация профиля. Напишите асинхронную функцию `initApp()`, которая создает папку `users`. Внутри создайте пустой файл `index.list` для будущего списка пользователей.

Задание 2: Безопасное чтение конфига. Напишите функцию `loadConfig()`, которая пытается прочитать файл `config.json`. Если файл найден — выведите "Конфиг загружен". Если возникла ошибка `ENOENT` — выведите "Используются настройки по умолчанию".

Задание 3: Обновление данных. Создайте функцию `saveProfile(username)`, которая асинхронно записывает имя пользователя в файл `users/current.txt`. При каждом новом вызове старое имя должно заменяться новым.

Задание 4: Перенос данных. Напишите скрипт, который:

- Асинхронно читает список файлов из папки `users`.
- Находит файл `current.txt`.
- Переименовывает (перемещает) его в папку `users/archive/backup.txt` (папку `archive` нужно предварительно создать асинхронно).

Задание 5: Массовое создание файлов. Дополнительное задание: Используя цикл и `await`, создайте в папке `users` 5 файлов с названиями `user1.tmp`, `user2.tmp` и так далее. Затем удалите их все одной асинхронной операцией (подумайте, как использовать `Promise.all` для удаления).